

Module MC-5: Keyword spotting features and training in Python

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 8, 2026

After this module you can:

- ▶ Name the six steps between the samples of a microphone and the input of a keyword spotting model, and say what each one does.
- ▶ Build the triangular filters of the Mel scale and explain why the low frequencies get more filters.
- ▶ Compute the cepstral coefficients of a clip of one second with numpy only.
- ▶ Train a small classifier on those coefficients and compare it with the same network on the log Mel energies and on the raw audio.

- ▶ A device listens to the room and must say one of four things at every moment: yes, no, **unknown**, or noise.
- ▶ The microphone gives 16000 values every second. No model of a microcontroller can read them.
- ▶ The pipeline of this module turns one second of audio into 650 values, and a small network turns those values into the class.

Key point: The pipeline runs on the device at every window. Its cost is part of the latency of the product.

- ▶ The sampling rate says how many values a second the microphone gives. 16 kHz is the usual rate for speech.
- ▶ The voice has no content above 8 kHz. The telephone system uses 8 kHz for that reason, and the CD uses 44.1 kHz because it keeps the music.
- ▶ The rate must be at least twice the highest frequency that the signal carries, or the samples cannot show it.

One second of audio at 16 kHz is 16000 values with 16 bits each. That is 32000 bytes for one second of sound.

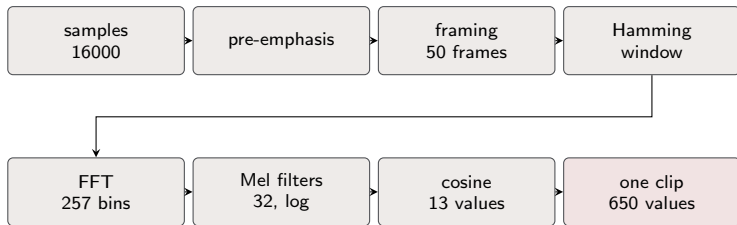
Source: Sampling rates and the 8 kHz of the voice, from "Machine Learning Systems", chapter "KWS Feature Engineering", section "Overview to Audio Signals".

- ▶ 16000 values for one second is a lot of input. A first layer with 16 filters of three values already has 768 parameters.
- ▶ A network must learn that a form of the spectrum means a word. It learns that from data, and the data is expensive.
- ▶ The same sound changes with the distance to the microphone, with the direction of the mouth, and with the room. Raw samples carry all of that, and the model has to remove it.
- ▶ Speech carries no more than 8 kHz, so 8000 of the 16000 samples of each second carry nothing.

Key point: Reduce the values first, with a method whose output you can name.

Source: The reasons for not using raw audio, from "Machine Learning Systems", chapter "KWS Feature Engineering", section "Why Not Raw Audio?".

The pipeline of the module



The numbers are those of the lab of this module. Every step is six lines of numpy, and every step runs on the device.

Step 1: the pre-emphasis

$$y[n] = x[n] - \alpha x[n - 1], \quad \alpha \approx 0.97$$

- ▶ The filter takes the difference between two consecutive samples.
- ▶ The high frequencies are small in the spectrum of a voice, so the difference lifts them. The spectrum is then flatter.
- ▶ The step is one subtraction and one multiplication per sample.
- ▶ The factor 0.97 is a convention. A value from 0.9 to 0.98 gives the same result on this task.

The first sample of a clip has no sample before it. The lab keeps it unchanged.

Source: The formula and the factor of the chapter, from "Machine Learning Systems", chapter "KWS Feature Engineering", section "Computing MFCCs".

Step 2: the framing

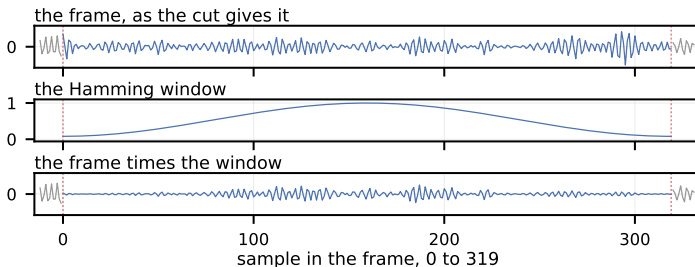
- ▶ The spectrum of one second says nothing about when the sound happened. So the clip is cut into short frames.
- ▶ A frame lasts 20 to 40 ms. Inside such a frame the frequencies of the signal stay about the same.
- ▶ The stride is the distance between two frames. The same value as the frame length means no overlap.

At 16 kHz, a frame of 20 ms is 320 samples. A clip of 1 s gives
 $1 + (16000 - 320)/320 = 50$ frames.

Source: The length of a frame, from "Machine Learning Systems", chapter "KWS Feature Engineering", section "Computing MFCCs". The count of 50 frames is the value of the lab of this module.

Step 3: the window

frame 38 of the clip yes: the largest jump at the two ends, 20 ms at 16 kHz



- ▶ A frame starts and ends in the middle of the sound. The jump at each end looks like a high frequency that is not there.
- ▶ The Hamming window multiplies the frame by a curve that is 1 in the middle and close to 0 at the two ends.
- ▶ It removes the jump, and it removes a little resolution. A longer frame resolves better and says less about the time.

Step 4: the FFT and the power

- ▶ The FFT of a frame of 320 values with 512 points gives 257 bins.
- ▶ The bins are 31.25 Hz apart, and bin 0 is the direct current.
- ▶ The power spectrum is the square of the magnitude:
 $P(f) = |X(f)|^2$. Squaring lifts the strong frequencies over the weak ones.

The frame is shorter than the FFT, so the FFT pads it with 192 zeros. The bins then sit closer than the frame can resolve, and the curve is smoother. The resolution stays that of 20 ms, which is 50 Hz.

Source: "Machine Learning Systems", chapter "KWS Feature Engineering", the steps "FFT" and the note on the power spectrum.

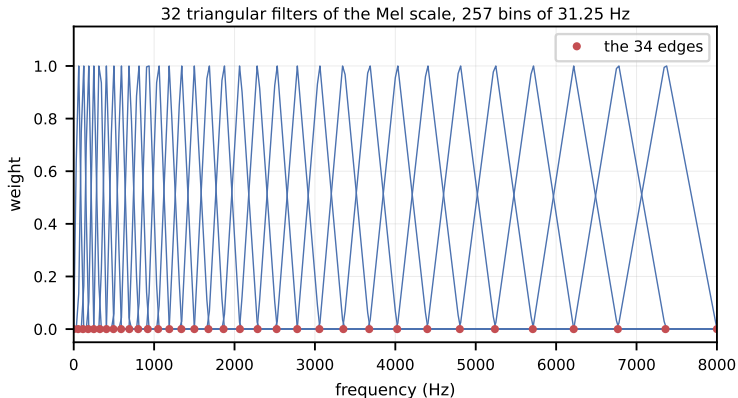
$$\text{mel}(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

- ▶ 1000 Hz on the linear scale is 1000 mel. 1000 Hz on the Mel scale is 1500 mel, and 3000 Hz is 2000 mel.
- ▶ The ear hears the difference between two low tones better than the difference between two high tones of the same width in hertz.
- ▶ So the filters of the bank are narrow at the bottom and wide at the top. With 32 filters from 0 to 8000 Hz, the first is about 60 Hz wide and the last about 660 Hz.

The number of filters is a choice. The chapter recommends 20 to 40 triangles.

Source: The Mel scale and the number of filters, from "Machine Learning Systems", chapter "KWS Feature Engineering", the steps "Mel Filter Banks" and the section "Computing MFCCs". The two widths are the value of the lab of this module.

Step 5: the filter bank and the logarithm



Each filter is a triangle that rises from 0 to 1 and falls back to 0. Two filters touch at their half point, so no frequency is lost. The energy of one filter is the sum of the power of the bins under it.

The logarithm comes next: it turns the wide dynamic range of audio into a range that a network can work with.

Step 6: the cosine transform

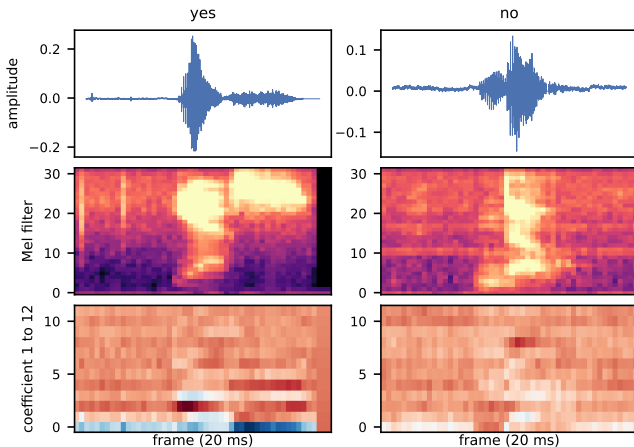
$$c[k] = \sum_{m=0}^{N-1} \log E[m] \cos \left(\frac{\pi k(2m+1)}{2N} \right)$$

- ▶ The cosine transform mixes the 32 log energies of a frame into 13 numbers.
- ▶ The first coefficient is the mean of the 32 values, so it holds the energy of the frame.
- ▶ The other coefficients describe the shape of the spectrum: the first ones the envelope, the last ones the fine structure.
- ▶ Mixing decorrelates the values, so the network sees numbers that do not repeat one another.

The chapter keeps the first 12 or 13 coefficients and drops the rest.

Source: "Machine Learning Systems", chapter "KWS Feature Engineering", the step "Discrete Cosine Transform (DCT)".

One second of audio in three forms



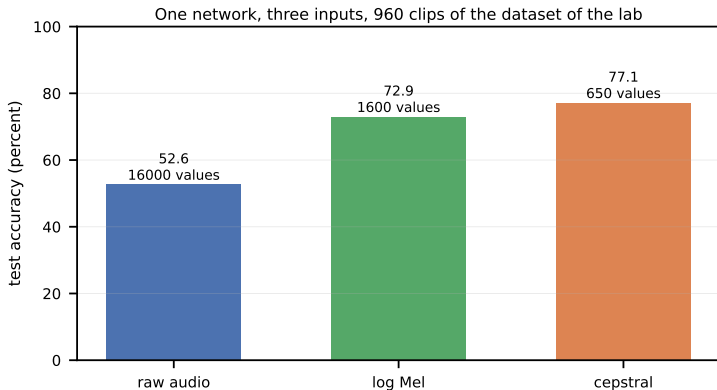
The top row is the waveform, the middle row the energy of the 32 filters for the 50 frames, and the bottom row the coefficient 1 to 12 of each frame. The first coefficient is not drawn: it is the energy.

What each step does to the number of values

After	Values per frame	What it is
The samples	16000	one second, no frame
The framing	320	20 ms of samples
The FFT	257	one value per bin
The Mel filters	32	the energy of each filter
The logarithm	32	the energy in a decibel scale
The cosine transform	13	the cepstral coefficients

- ▶ A clip of 1 s has 50 frames, so it gives $50 \times 13 = 650$ values instead of 16000. The pipeline divides the number of values by 24.6.
- ▶ Only two steps reduce the number of values: the filters and the cosine transform.

The cepstrum against the spectrogram



One network, three inputs, the same 768 training clips. The cepstrum wins on all three counts: the fewest values, the fewest parameters, and the best test accuracy.

The raw audio needs 25 times more values than the cepstrum, 142 times more parameters, and it does not learn with this little data.

When the spectrogram is the better choice

The cepstrum is stronger for

- ▶ speech and keywords,
- ▶ the identity of a speaker,
- ▶ the emotion of a voice,
- ▶ a device with little memory.

The spectrogram is stronger for

- ▶ music and the instruments in it,
- ▶ rain, wind, and traffic,
- ▶ the calls of birds,
- ▶ finding a click in a recording.

The cepstrum keeps the envelope of the spectrum and drops its fine structure, so it is stable against the noise and against a change of loudness. A task that lives in the fine structure needs the whole spectrum.

Source: "Machine Learning Systems", chapter "KWS Feature Engineering", the sections "MFCCs are particularly strong for" and "Spectrograms or MFEs are often more suitable for".

1. One second of audio at 16 kHz is 16000 values. The model must not read them.
2. Six steps: the pre-emphasis, the framing, the window, the FFT, the Mel filters with the logarithm, and the cosine transform.
3. The Mel scale gives the low frequencies more filters, because the ear resolves them better.
4. The cosine transform is the only step that reduces the number of values by a large factor: 32 energies become 13 coefficients.
5. Choose between the cepstrum and the spectrogram from the task, and check the choice with a measurement.

A device samples at 16 kHz, cuts the signal into frames of 20 ms with the same stride, uses an FFT of 512 points, 32 triangular filters of the Mel scale, and keeps 13 cepstral coefficients.

1. How many values does one second of audio give before the pipeline, and after it?
2. How many multiply-accumulate operations does the first convolution of the lab network need for one clip, with 16 filters of 3 frames and 13 input values?
3. Name the step that reduces the number of values the most.

1. Before: 16000 samples. After: a clip of 1 s has $1 + (16000 - 320)/320 = 50$ frames, and each frame gives 13 values, so $50 \times 13 = 650$ values. The pipeline divides the number of values by 24.6.
2. A convolution with 3 frames and no padding leaves $50 - 2 = 48$ positions. Each position needs $3 \times 13 \times 16 = 624$ multiply-accumulate operations, so the layer needs $48 \times 624 = 29952$ operations for one clip.
3. The cosine transform, which turns the 32 log energies of a frame into 13 coefficients. The FFT raises the number of values from 320 to 257 before the filters, and the filters keep 32.

Part 2 assumes the network of the lab, with valid convolutions. A convolution with a padding of 'same' would keep the 50 positions and need 31200 operations.

- ▶ **Goal.** You compute the cepstral coefficients of a clip of one second with numpy only, and you train a small classifier on them, with no tool of Edge Impulse Studio.
- ▶ **Deliverable.** The table of section 10 of the notebook, copied into `report.md`, and the answers of its three questions.

Part	Content	Time
A	The clips, and the triangular filters of the Mel scale	12 min
B	The cepstral coefficients, and the plot of one clip	13 min
C	The classifier, and its number of parameters	8 min
D	Three inputs for the same task	12 min

- ▶ The script `host/get_keywords.py` downloads the file `keywords2.zip` of Edge Impulse (139 MB) into the folder `downloads/`, one time. Then it writes 240 clips of each of the four classes into `keywords/clips/`.
- ▶ The four classes are `no`, `noise`, `unknown`, and `yes`. Each clip has 1 s of audio at 16 kHz.
- ▶ The notebook reads the amplitudes from text files, so it needs no audio package.

Class	Clips	Training	Test
no	240	192	48
noise	240	192	48
unknown	240	192	48
yes	240	192	48

The split is random, so a clip of one speaker can be in both sets. The notebook says so in section 6.

File	Content
<code>mfcc_training.ipynb</code>	The notebook. It has four tasks.
<code>host/get_keywords.py</code>	Downloads the dataset.
<code>report.md</code>	The report to hand in.
<code>solutions/</code>	The notebook with all tasks complete and with its output, and an example report.
<code>TEST_NOTES.md</code>	The code status and the steps to run the lab.

- ▶ Section 2 of the notebook gives the functions of the pipeline that need no decision from you. Two of the four tasks use them.
- ▶ Section 6 caches the features in the file `features.npz`, so the second run of the notebook is fast.

1. Get the dataset, one time, from the folder of the lab:

```
python3 host/get_keywords.py
```

2. Start Jupyter from the folder of the lab, and run sections 0 and 1.
3. **Task 1.** Write `mel_filters(n_filters, n_bins, f_min, f_max, sample_rate)`: the matrix of the 32 triangular filters of the Mel scale.
4. Run the cell after the task.

You see the peak of the first filter at bin 2 and of the last one at bin 236. The first filter is 3 bins wide and the last one 40 bins wide. That is the Mel scale: narrow at the bottom, wide at the top.

1. **Task 2.** Write `mfcc(clip, filters=None)`: the pre-emphasis, the framing, the Hamming window, the FFT, the filters, the logarithm, and the cosine transform.
2. Run the cell after the task. Then run section 5.
3. Look at the three panels. Write in the report what the middle panel shows and what the bottom panel shows.
4. Run section 6. Write the run time of the features in the report.

The notebook prints the mean of the first coefficient, which is -13.071 , and the mean of the last one, which is 0.064 . The first coefficient is large because it holds the energy of the frame, and the logarithm of a small power is a large negative number.

1. **Task 3.** Write `make_model(n_frames, n_features)`: two `Conv1D` layers with 16 and with 32 filters of 3 frames, each with a max pooling of 2, then a `Dense` layer with 4 outputs.
2. Run the cell after the task. The check calculates the expected number of parameters itself, with the length that each convolution and each pooling layer leaves.
3. Run the next cell to train the model.

the input has 50 frames of 13 values, and 11 frames after the two convolutions and the two pooling layers
parameters 3620 (expected 3620), outputs 4
the cepstral coefficients: 650 values per clip, 3620 parameters,
test accuracy 77.1 percent

1. **Task 4.** Before you run the next cell, complete `values_per_clip` with the number of values that one clip gives for each of the three inputs, and write in `best_input` the input that you expect to win.
2. Run the next cell and section 10, and copy the table into the report.
3. Answer the three questions of the notebook.

The three lines of the notebook: the raw audio has 16000 values and 513380 parameters, the log Mel spectrogram 1600 values and 4532 parameters, and the cepstral coefficients 650 values and 3620 parameters.

- ▶ The notebook prints complete for tasks 1, 2, 3, and 4.
- ▶ The table of the report has three rows, each with the number of values, the number of parameters, and a measured test accuracy.
- ▶ The three questions of the report have answers with numbers from your own run.
- ▶ The Decision Log names one decision and one number.

Key point: Question of the Decision Log: the device must say one word every 500 ms on a battery. Which of the three inputs do you put in the firmware, and which number of your table decides it?

Problem	Fix
The notebook stops with No clip in the folder	Run <code>host/get_keywords.py</code> first. The download is 139 MB.
Task 1 says that the last filter must reach bin 205	A bin is $\text{frequency} * \text{nfft} / \text{sample_rate}$, and <code>nfft</code> is $2 * (\text{n_bins} - 1)$.
Task 2 says that the answer needs 50 rows	A clip of 16000 samples at a stride of 320 gives 50 frames. Return one row for each.
Task 3 says that the model raises an error	A layer is missing, or a layer was added with a tensor instead of with its input.
The plot of section 5 is empty	Delete <code>features.npz</code> and run section 6 again.
The notebook is slow	The first run computes the features of 960 clips. The file <code>features.npz</code> makes the next run fast.

The README of the lab folder has the full list of the checks of the notebook and the run time.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. github.com/Mjrovai/XIAO-ESP32S3-Sense, Apache-2.0.